Kernel Module:

1.Kernel modules are pieces of code that can be loaded and unloaded into the kernel upon demand. They extend the functionality of the kernel without the need to reboot the system.
2.Custom codes can be added to Linux kernels via two methods:

i. The basic way is to add the code to the kernel source tree and recompile the kernel.

ii. A more efficient way is to do this is by adding code to the kernel while it is running. This process is called loading the module, where module refers to the code that we want to add to the kernel.

3.Since we are loading these codes at runtime and they are not part of the official Linux kernel, these are called loadable kernel module (LKM), which is different from the "base kernel". Base kernel is located in /boot directory and is always loaded when we boot our machine whereas LKMs are loaded after the base kernel is already loaded. Nonetheless, these LKM are very much part of our kernel and they communicate with base kernel to complete their functions.

4.LKMs can perform a variety of task, but basically they come under three main categories:

i. Device driver

ii. Filesystem driver

iii. System calls.

Key points of Kernel Module:

1.Modularity and Flexibility: They allow for a more modular and flexible kernel architecture, enabling the addition or removal of specific functionalities as needed.

2. Dynamic Loading and Unloading: Kernel modules can be loaded into the kernel (using insmod) and unloaded (using rmmod) without restarting the system.

3.Kernel Extension: Kernel modules allow the kernel to be extended with new functionality without requiring a full kernel recompilation.

Process Management:

A process means **program in execution**. It generally takes an input, processes it and gives us the appropriate output. There are basically 2 types of processes:

i. Foreground processes: Such kind of processes are also known as **interactive processes**. These are the processes which are to be executed or initiated by the user or the programmer, they can not be initialized by system services. Such processes take input from the user and return the output. While these processes are running we can not directly initiate a new process from the same terminal.

ii. Background processes: Such kind of processes are also known as **non-interactive processes**. These are the processes that are to be executed or initiated by the system itself or by users, though they can even be managed by users. These processes have a unique PID or process if assigned to them and we can initiate other processes within the same terminal from which they are initiated.

Process Creation:

1.Process are typically created by the OS or another process the most common or the most important system calls for creating process in Linux is fork() which creates a new process by duplicating the parent process.

2.After the process is created it can execute its instruction a common wait to run a program is exec() after a fork().

i. fork(): Creates a new child process by duplicating the parent process. The child gets a unique Process ID (PID). Both processes execute the same code but with different execution flows.

ii. exec(): Replaces the current process with a new program. Does not create a new process but changes the existing one. Multiple variants exist (execl(), execv(), execvp(), etc.).

iii. wait(): Used by the parent process to wait for the child to finish execution. Prevents zombie processes.

Process Scheduling:

1.Process scheduling is the activity of the process manager that handles the removal of the running process from the CPU and the selection of another process based on a particular strategy. Throughout its lifetime, a process moves between various **scheduling queues**, such as the ready queue, waiting queue, or devices queue.

2.**Categories of Scheduling:** Scheduling falls into one of two categories->

i. Non-Preemptive: In this case, a process's resource cannot be taken before the process has finished running. When a running process finishes and transitions to a waiting state, resources are switched.

ii. Preemptive: In this case, the OS can switch a process from running state to ready state. This switching happens because the CPU may give other processes priority and substitute the currently active process for the higher priority process.

iii. I/O Schedulers: I/O schedulers are in charge of managing the execution of I/O operations such as reading and writing to discs or networks. They can use various algorithms to determine the order in which I/O operations are executed, such as FCFS (First-Come, First-Served) or RR (Round Robin).

3.Priority: Processes are assigned priority and the scheduler tries to allocate resources based on those priorities.

Long Term Scheduler	Short Term Schedular	Medium Term Scheduler
It is a job scheduler	It is a CPU scheduler	It is a process-swapping scheduler.
The slowest scheduler.	Speed is the fastest among all of them.	Speed lies in between both short and long-term schedulers.
It controls the degree of multiprogramming	It gives less control over how much multiprogramming is done.	It reduces the degree of multiprogramming.
It is barely present or nonexistent in the time-sharing system.	It is a minimal time-sharing system.	It is a component of systems for time sharing.
It can re-enter the process into memory, allowing for the continuation of execution.	It selects those processes which are ready to execute	It can re-introduce the process into memory and execution can be continued.

Process state:

In Linux, a process is an instance of executing a program or command. While these processes exist, they'll be in one of the five possible states:

1.Running or Runnable (R): When a new process is started, it'll be placed into the running or runnable state. In the running state, the process takes up a CPU core to execute its code and logic. If the process is pre-empted then, the process will be placed into a run queue, and its state is now a runnable state waiting for its turn to execute.

2.Uninterruptible Sleep (D): The process is waiting for a critical event (e.g., I/O operation) and cannot be interrupted. Waits for the resources to be available before it moves into a runnable state and doesn't react to any signals.

3.Interruptable Sleep (S): The process is waiting for an event and can be woken up (e.g., waiting for input or reacting to the signal). A process enters the *S* state, or *Interruptible Sleep*, when it waits for an event or condition that is not directly related to an I/O operation.

4.Stopped (T): The process is paused or killed in this state. It can be resumed using SIGCONT or killed using SIGKILL.

5.Zombie (Z): The process has finished execution but is waiting for the parent process to collect its exit status.

Process Termination:

1.Whenever the process finishes executing its final statement and asks the operating system to delete it by using exit() system call.

2.At that point of time the process may return the status value to its parent process with the help of wait() system call.

3.All the resources of the process including physical and virtual memory, open files, I/O buffers are deallocated by the operating system.

4.Reasons for process termination:

- The child exceeds its usage of resources that it has been allocated.
- The task that is assigned to the child is no longer required.
- The parent is exiting and the operating system does not allow a child to continue if its parent terminates.
- **Time slot expired** – When the process execution does not complete within the time quantum, then the process gets terminated from the running state. The CPU picks the next job in the ready queue to execute.
- **Memory bound violation** – If a process needs more memory than the available memory.
- **I/O failure** – When the operating system does not provide an I/O device, the process enters a waiting state.
- **Process request** – If the parent process requests the child process about termination.

5.Some systems, including VMS, do not allow a child to exist if its parent has terminated. In such systems, if a process terminates either normally or abnormally, then all its children have to be terminated. This concept is referred to as cascading termination.

Process Signals:

Signals are used to communicate with processes, allowing them to be stopped, killed, or handled in various ways.

1.SIGKILL(9): Immediately terminates a process (cannot be ignored or caught).

2.SIGTERM(15): Gracefully terminates a process (can be caught and handled).

3.SIGSTOP(19): Stops a process (cannot be ignored).

4.SIGCONT(18): Resumes a stopped process.

PCB:

1.A Process Control Block (PCB) is a data structure used by the operating system to manage information about a process. The process control keeps track of many important pieces of information needed to manage processes efficiently.

2.

- **Pointer:** It is a stack pointer that is required to be saved when the process is switched from one state to another to retain the current position of the process.
- **Process state:** It stores the respective state of the process.
- **Process number:** Every process is assigned a unique id known as process ID or PID which stores the process identifier.
- **Program counter:** Program Counter stores the counter, which contains the address of the next instruction that is to be executed for the process.
- **Register:** Registers in the PCB, it is a data structure. When a process is running and it's time slice expires, the current value of process specific registers would be stored in the PCB and the process would be swapped out. When the process is scheduled to be run, the register values is read from the PCB and written to the CPU registers. This is the main purpose of the registers in the PCB.
- **Memory limits:** This field contains the information about memory management system used by the operating system. This may include page tables, segment tables, etc.
- **List of Open files:** This information includes the list of files opened for a process.

Parent and Child Process:

1.Child Process: A child process is created by a parent process in an operating system using a fork() system call. A child process may also be known as subprocess or a subtask.

- A child process is created as a copy of its parent process.
- The child process inherits most of its attributes.
- If a child process has no parent process, then the child process is created directly by the kernel.
- If a child process exits or is interrupted, then a SIGCHLD signal is sent to the parent process to inform about the termination or exit of the child process.

2.Parent Process: All the processes are created when a process executes the fork() system call except the startup process. The process that executes the fork() system call is the parent process. A parent process is one that creates a child process using a fork() system call. A parent process may have multiple child processes, but a child process only one parent process.

On the success of a fork() system call:

- The Process ID (PID) of the child process is returned to the parent process.
- 0 is returned to the child process.

On the failure of a fork() system call,

- -1 is returned to the parent process.
- A child process is not created.

Daemon Process:

1.A **daemon process** is a background process that runs independently of any user control and performs specific tasks for the system. Daemons are usually started when the system starts, and they run until the system stops.

2.A daemon process typically performs system services and is available at all times to more than one task or user. Daemon processes are started by the root user or root shell and can be stopped only by the root user.

- A daemon process runs in the background without direct user interaction.
- It is often started automatically when the system boots.
- Daemons usually detach from their parent processes and run independently.
- example - the "qdaemon" process, "sendmail" daemon etc.

3.They are managed by the tools like system and init.

GNU Utilities in Linux: GNU Utilities are a collection of essential command-line tools developed by the GNU Project to provide a Unix-like operating system environment. They form the core of many Linux distributions. They provide the core functionality needed for Linux systems. Many are improved versions of Unix commands. They allow automation, scripting, and system administration tasks.

File and directory management:

1.ls: List files and directories

2.cp: Copy files

3.mv: Move or rename files

4.rm: Remove files or directories

5.mkdir: Create directories

6.find: Used to search for files and directories based on name, size, type, modification time, etc.

7.stat: Used to display detailed information about a file or directory, including size, permissions, timestamps, and more.

Text processing utilities:

Command	Description
`cat`	Displays the contents of a file. Can also concatenate multiple files.
`tac`	Displays file contents in reverse order, from last to first line.
`head`	Displays the first 10 lines of a file (default) or a specified number of lines.
`tail`	Displays the last 10 lines of a file (default) or a specified number of lines.
`cut`	Extracts specific columns or fields from a file based on a delimiter.
`paste`	Merges lines from multiple files side by side, joining them with a tab or specified delimiter.
`sort`	Sorts lines of a text file in alphabetical or numerical order.
`uniq`	Removes or displays duplicate lines from sorted input.
`wc`	Counts the number of lines, words, and characters in a file.
`grep`	Searches for specific patterns in a file using regular expressions.
`sed`	A stream editor used for searching, replacing, deleting, and modifying text.
`awk`	A powerful text processing tool for pattern scanning, column extraction, and data manipulation.

Process and System monitoring:

Command	Description
`ps`	Displays information about active processes.
`top`	Shows real-time system resource usage and running processes.
`htop`	Interactive process viewer with better visualization than `top`.
`kill`	Sends a signal to terminate a specific process by its PID.
`pkill`	Kills processes by name instead of PID.
`nice`	Starts a process with a specific priority.
`renice`	Changes the priority of a running process.
`uptime`	Shows system uptime, number of users, and load average.
`free`	Displays system memory usage (RAM and swap).
`df`	Shows disk space usage of file systems.
`du`	Displays the size of directories and files.

User and Permission management:

Command	Description
`whoami`	Displays the current logged-in user.
`who`	Shows a list of all logged-in users.
`id`	Displays the user ID (UID), group ID (GID), and group memberships.
`chmod`	Changes file or directory permissions.
`chown`	Changes the owner of a file or directory.
`chgrp`	Changes the group ownership of a file or directory.
`passwd`	Changes the user password.

Networking Utilities:

Command	Description
`ping`	Checks network connectivity to a host by sending ICMP echo requests.
`wget`	Downloads files from the internet via HTTP, HTTPS, and FTP.
`curl`	Transfers data from or to a server using various protocols (HTTP, FTP, etc.).
`netstat`	Displays network connections, routing tables, and interface statistics (deprecated, use `ss`).
`ss`	Shows detailed network socket information (replacement for `netstat`).
`ip`	Displays and manipulates IP addresses, routing, and network interfaces (replacement for `ifconfig`).
`nslookup`	Queries DNS servers to resolve domain names to IP addresses.
`traceroute`	Traces the path packets take to a destination, showing each hop.

Linux Console: The Linux Console is a text-based interface that allows users to interact with the operating system by executing commands. It provides direct access to the system without a graphical interface.

Key Feature of Linux Console:

1.CLI: It allows users to execute commands directly.

2.No Graphical Interface.

3.Works without desktop environment using only text-based command.

4.Light weight and fast uses minimal system resources making it ideal for server and low-end system.

5. Direct hardware access useful for system recovery, trouble shooting and boot time system configurations.

Root the system administrator login: The root user in Linux is the most privileged account, often referred to as the superuser or system administrator. This account has unrestricted access to all system resources, files, and configurations, allowing it to perform any administrative task.

Key Characteristics of the Root User:

1.Full System Control: The root user can modify critical system files, install and remove software, configure system settings, and manage hardware.

2.User and Permission Management: Root has the authority to create, modify, or delete user accounts, assign permissions, and control access to system resources.

3.Superuser privileges: The superuser (root user) in Linux has the highest level of control over the system. It has unrestricted access to all files, commands, and system resources. This account is used for performing administrative tasks that require elevated permissions. Such as filesystem access, user management, process control, software installation, etc.

4.Critical Operation: The root user is responsible for performing essential system operations that regular users cannot execute. These operations include:

i. System file management.

ii. User and group management.

iii. Process and service management.

iv. Software updates and installation.

v. Firewall and security management.

Common command used by Admin:

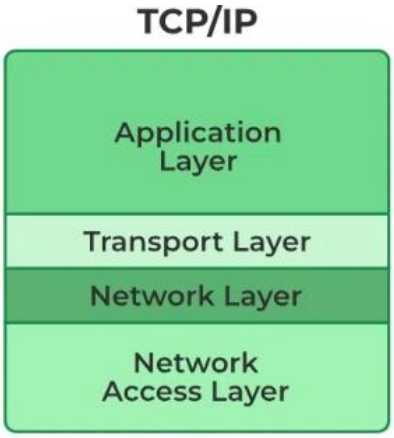
Command	Description	Example
`sudo`	Executes a command with root privileges without logging in as root.	`sudo apt update` (Update system packages)
`su`	Switches to another user account, commonly used to become root.	`su -` (Switch to root user)
`chmod`	Changes file permissions (read, write, execute) for users, groups, and others.	`chmod 755 script.sh` (Give execute permission to all, write only to owner)
`chown`	Changes the ownership of a file or directory.	`chown user:group file.txt` (Change file ownership)

TCP/IP basics:

The TCP/IP model is a fundamental framework for computer networking. It stands for Transmission Control Protocol/Internet Protocol, which are the core protocols of the Internet. This model defines how data is transmitted over networks, ensuring reliable communication between devices. It consists of four layers: the Network Layer, the Internet Layer, the Transport Layer, and the Application Layer. Each layer has specific functions that help manage different aspects of network communication, making it essential for understanding and working with modern networks.

TCP/IP was designed and developed by the Department of Defense (DoD) in the 1960s and is based on standard protocols. The TCP/IP model is a concise version of the OSI model.

TCP/IP Model:



1. Data Creation (Application Layer → Transport Layer):
   - A user sends data using an application (e.g., a web browser request using HTTP).
   - The Application Layer passes the data to the Transport Layer.
2. Data Segmentation (Transport Layer → Internet Layer):
   - The Transport Layer (TCP/UDP) breaks the data into smaller chunks called segments.
   - TCP adds a sequence number for reliability, while UDP sends it without ordering.
   - The segments are forwarded to the Internet Layer.
3. Addressing & Routing (Internet Layer → Network Access Layer):
   - The Internet Layer assigns an IP address to each data segment.
   - It determines the best path for the data and forwards it to the Network Access Layer.
4. Physical Transmission (Network Access Layer → Network Medium):
   - The Network Access Layer converts the data into frames and adds MAC (hardware) addresses.
   - The frames are physically transmitted as electrical signals or radio waves over Ethernet or Wi-Fi.
5. Data Travel Across the Network:
   - The data reaches the destination device through multiple routers and networks.
6. Receiving Device (Reverse Process: Bottom to Top):
   - Network Access Layer receives the signals and reconstructs frames.
   - Internet Layer extracts the IP address and verifies if it belongs to the device.
   - Transport Layer reorders TCP segments (if necessary) and reassembles the message.
   - Application Layer processes the data and displays it to the user (e.g., webpage loads in a browser).

Key Points of TCP:

1.Connection-Oriented: Establishes a connection between sender and receiver before data transmission.

2.Reliable Data Transfer: Ensures all data packets are delivered correctly and in order.

3.Error Detection & Correction: Uses checksums and acknowledgments to detect and correct errors.

4.Packet Sequencing: Assigns sequence numbers to packets to ensure correct ordering.

5.Flow Control: Prevents data overload using mechanisms like sliding window.

6.Congestion Control: Adjusts transmission speed based on network conditions (e.g., TCP slow start).

Key Points of IP:

1.Routing packets: Determines the best path for packets to reach their destination. Data is broken into packets and transmitted independently.
2.Unreliable Delivery: No guarantee of packet delivery, order, or error correction (handled by TCP if needed).
3. Pv4 and IPv6: Supports two main versions:

- IPv4 (32-bit addressing, most widely used).
- IPv6 (128-bit addressing, solves IPv4 exhaustion).

FTP (file transfer protocol):

FTP (file transfer protocol) is an internet protocol that is used for transferring files between client and server over the internet or a computer network. It is similar to other internet protocols like SMTP which is used for emails and HTTP which is used for websites. FTP server enables the functionality of transferring files between server and client. A client connects to the server with credentials and depending upon the permissions it has, it can either read files or upload files to the server as well.

How does the FTP server work?

FTP server facilitates the transfer of files between client and server. You can either upload a file to a server or download a file from the server. A client makes two types of connections with the server, one for giving commands and one for transferring data. The client issues the command to the FTP server on port 21, which is the command port for FTP. For transferring data, a data port is used.

There are two types of connection modes for transferring data:

1.Active mode: In Active mode, the client opens a port and waits for the server to connect to it to transfer data. The server uses its port 20 to connect to the client for data transfer. Active mode is not set by default in most of the FTP clients because most firewalls block the connections which are initiated from outside, in this case, the connection initiated by our FTP server. To use this, you have to configure your firewall.

2.Passive mode: In this, when a client requests a file from the server, the server opens a random port and tells the client to connect to that port. In this case, the connections are initiated by the client and this also solves the firewall issues. Most of the FTP clients use passive mode by default.

Security issues in FTP:

1.Plaintext Credentials: FTP transmits usernames and passwords in plaintext, making them easy to intercept.

2.Data Exposure: Files and commands are also sent without encryption, leading to potential data leaks.

3.Man-in-the-Middle (MITM) Attacks: Attackers can intercept and modify data during transmission.

Secure Alternatives to FTP:

1.SFTP (Secure FTP): Uses SSH to encrypt FTP traffic.

2.FTPS (FTP Secure): Adds TLS/SSL encryption to standard FTP.

Berkeley Commands:

It refers to a set of command that are part of Berkeley Software Distributions (BSD) which is an early version of UNIX's OS many of this command are part of the networking utilities in BSD and have been widely adopted in UNIX like system including Linux. This commands are often associated with network configuration and troubleshooting task.

Command	Description
`ifconfig`	Displays or configures network interface settings (IP address, MAC address, etc.). Mostly replaced by `ip` in modern Linux.
`netstat`	Shows network connections, routing tables, and interface statistics. Replaced by `ss` in newer systems.
`route`	Displays or modifies the IP routing table. Replaced by `ip route` in modern systems.
`ping`	Sends ICMP echo requests to test network connectivity and measure response time.
`traceroute`	Tracks the path packets take to reach a destination, showing each hop along the way.
`nslookup`	Queries DNS servers to get domain name or IP address information.
`hostname`	Displays or sets the system's hostname.
`arp`	Shows or modifies the ARP (Address Resolution Protocol) cache, mapping IP addresses to MAC addresses.

Partitioning in Linux:

Partitioning in Linux is the process of dividing a physical storage device (like an HDD or SSD) into separate sections called partitions. Each partition can function as an independent storage unit, making it easier to manage files, install multiple operating systems, or separate user data from system files.

Types of partitioning:

1.Primary Partition: A bootable partition that can contain an OS. A disk can have up to 4 primary partitions.

2.Extended Partition: A special type of primary partition that can contain multiple logical partitions.

3.Logical Partition: A sub-partition inside an extended partition. You can create more than 4 logical partitions if needed.

Partitioning Schemes:

1.MBR (Master Boot Record): A traditional partitioning scheme with a 2TB disk size limit and support for only 4 primary partitions.

2.GPT (GUID Partition Table): Supports unlimited partitions (typically up to 128). No 2TB limit. Required for UEFI-based systems.

Common File System:

1.EXT4 (Fourth Extended File System): This is the most widely used file system in Linux, known for its performance, reliability, and efficiency. It is the default file system in most Linux distributions, including Ubuntu, Debian, and Fedora.

Journaling: Uses a journal to log changes before committing them to disk, reducing the risk of data corruption after a crash or power failure.

Supports Large File Sizes: Maximum file size: 16TB, Maximum partition size: 1EB (Exabyte)

2.XFS: This is a high-performance journaling file system designed for large storage systems, high-speed data access, and scalability. It is commonly used in enterprise servers, NAS (Network Attached Storage), and databases.

High Performance & Scalability: Handles large files (up to 8 exabytes) efficiently. Optimized for parallel I/O operations, making it ideal for high-performance workloads.

Journaling for Data Integrity: Uses metadata journaling, reducing the risk of corruption after crashes.

3.BTRFS (B-Tree File System): BTRFS (pronounced as "Butter FS" or "B-Tree FS") is a modern copy-on-write (CoW) file system designed for high performance, scalability, and advanced storage management. It is commonly used in enterprise environments, NAS devices, and modern Linux distributions like Fedora and openSUSE**.** Supports snapshots, compression and RAID like functionality.

RAID: BTRFS has built-in RAID like functionality, meaning you don’t need internal tools, instead BRTFS can manage multiple disks directly with its own RAID like function. RAID is Reductant Array of Independent Disks can be managed by using MDADM (Multiple Device Admin) a RAID software tools.